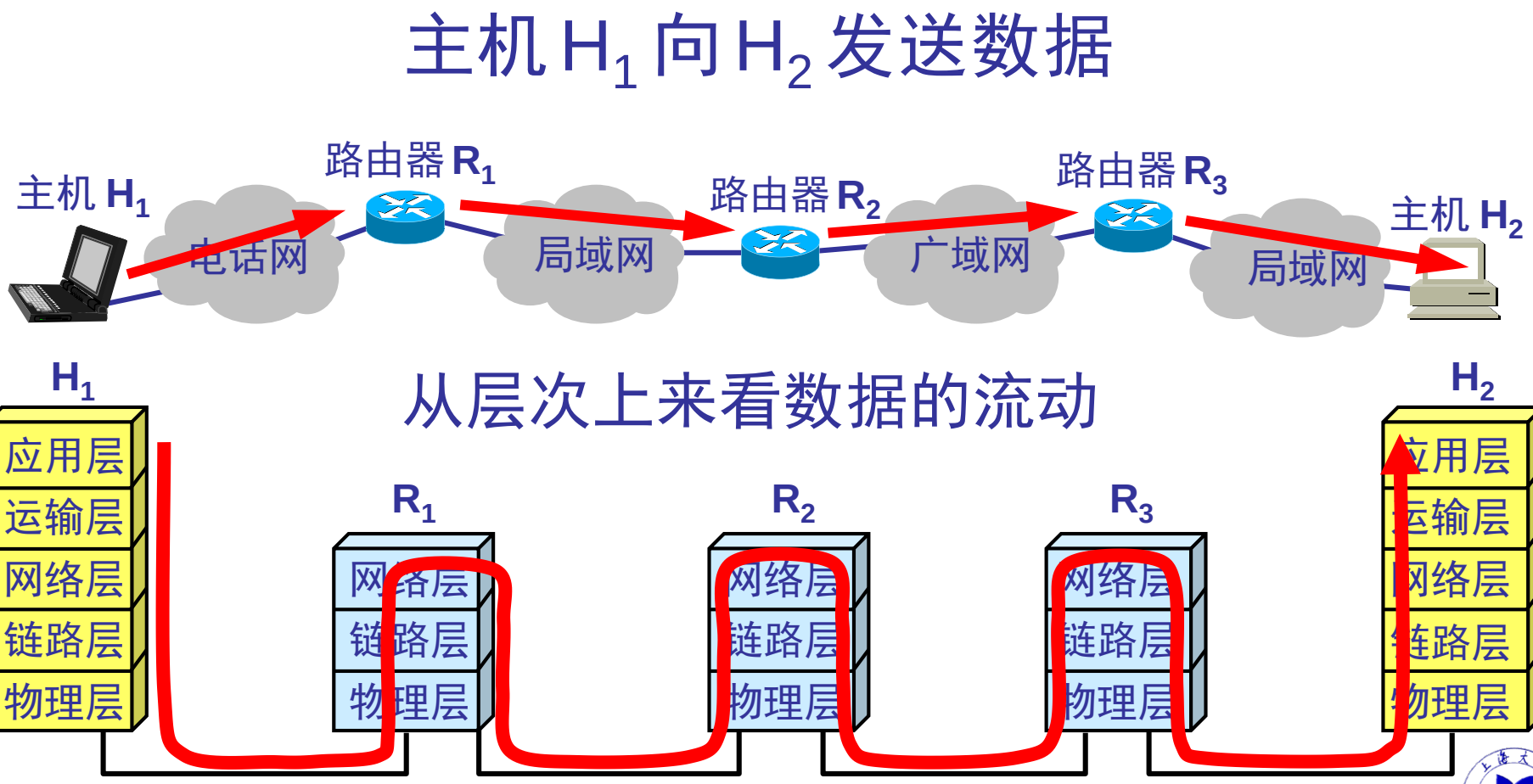
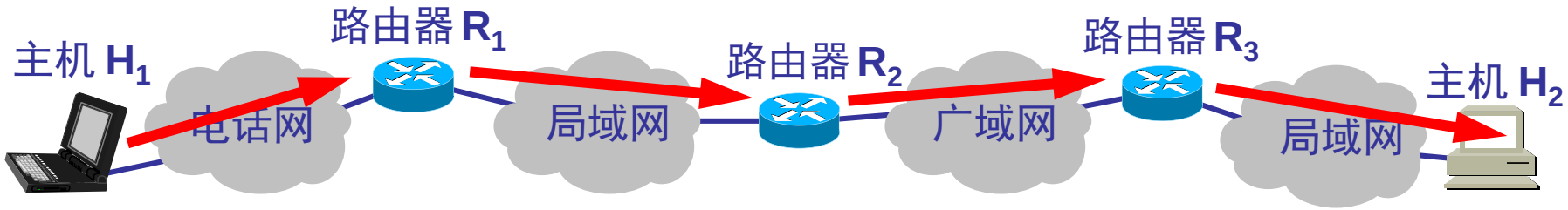
数据链路层的简单模型

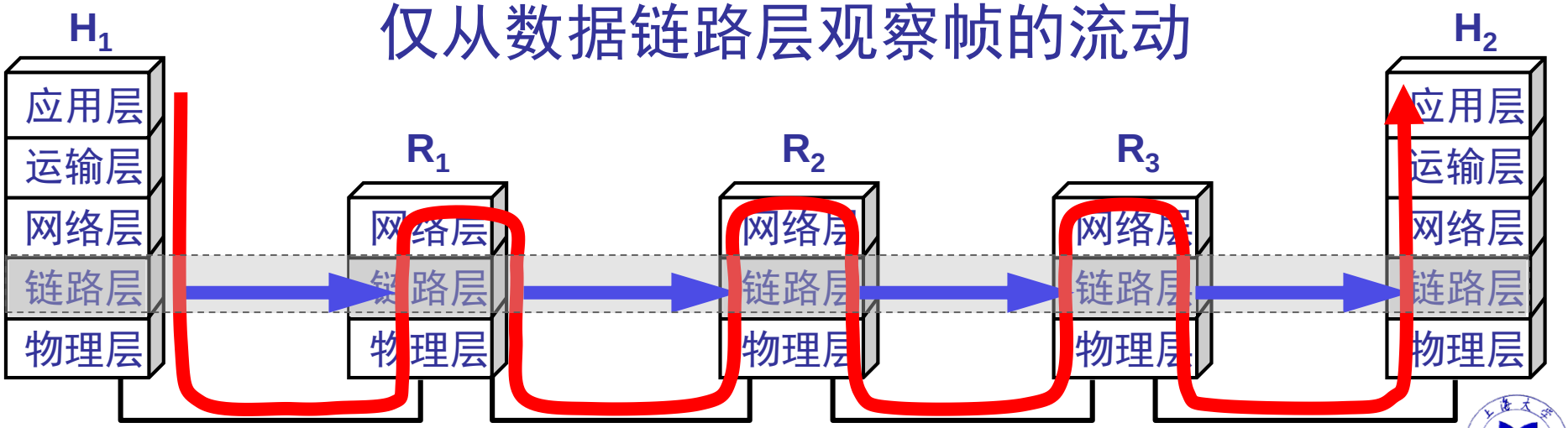

主机 H1 向 H2 发送数据
主机 H1
路由器 R1
路由器 R2
路由器 R3
主机 H2
电话网
局域网
广域网
局域网
从层次上来看数据的流动
H1
R1
R2
R3
H2
应用层
运输层
网络层
链路层
物理层

数据链路层的简单模型(续)

主机 H_1 向 H_2 发送数据



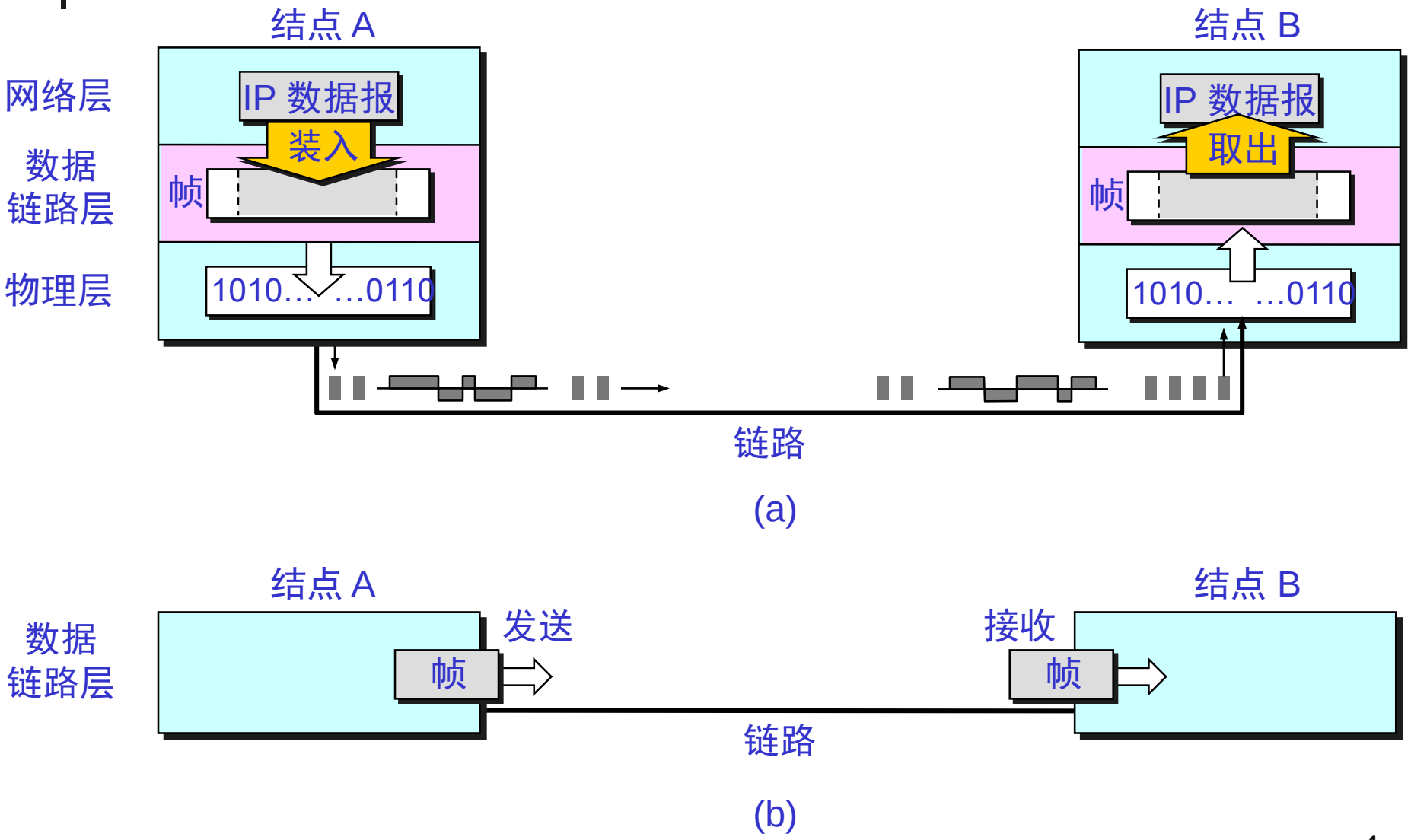
仅从数据链路层观察帧的流动



3.1 数据链路层的基本问题

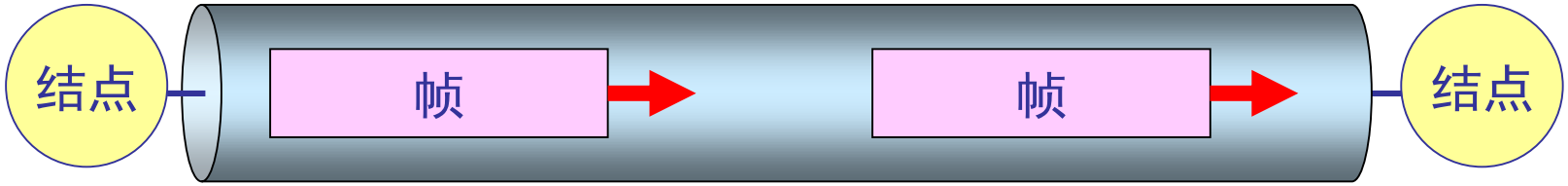
- 链路(link)是一条无源的点到点的物理线路段，中间没有任何其他的交换结点。
  - 一条链路只是一条通路的一个组成部分。
- 数据链路(data link) 除了物理线路外，还必须有通信协议来控制这些数据的传输。若把实现这些协议的硬件和软件加到链路上，就构成了数据链路。
  - 现在最常用的方法是使用网络适配器（即网卡）来实现这些协议的硬件和软件。
  - 一般的适配器都包括了数据链路层和物理层这两层的功能。

数据链路层传送的是帧


结点 A
结点 B
网络层
数据
链路层
物理层
IP 数据报
装入
帧
1010…0110
IP 数据报
取出
帧
1010…0110
链路
(a)
结点 A
结点 B
数据
链路层
发送
帧
接收
帧
链路
(b)

数据链路层像个数字管道

- 常常在两个对等的数据链路层之间画出一个数字管道，而在这条数字管道上传输的数据单位是**帧**。



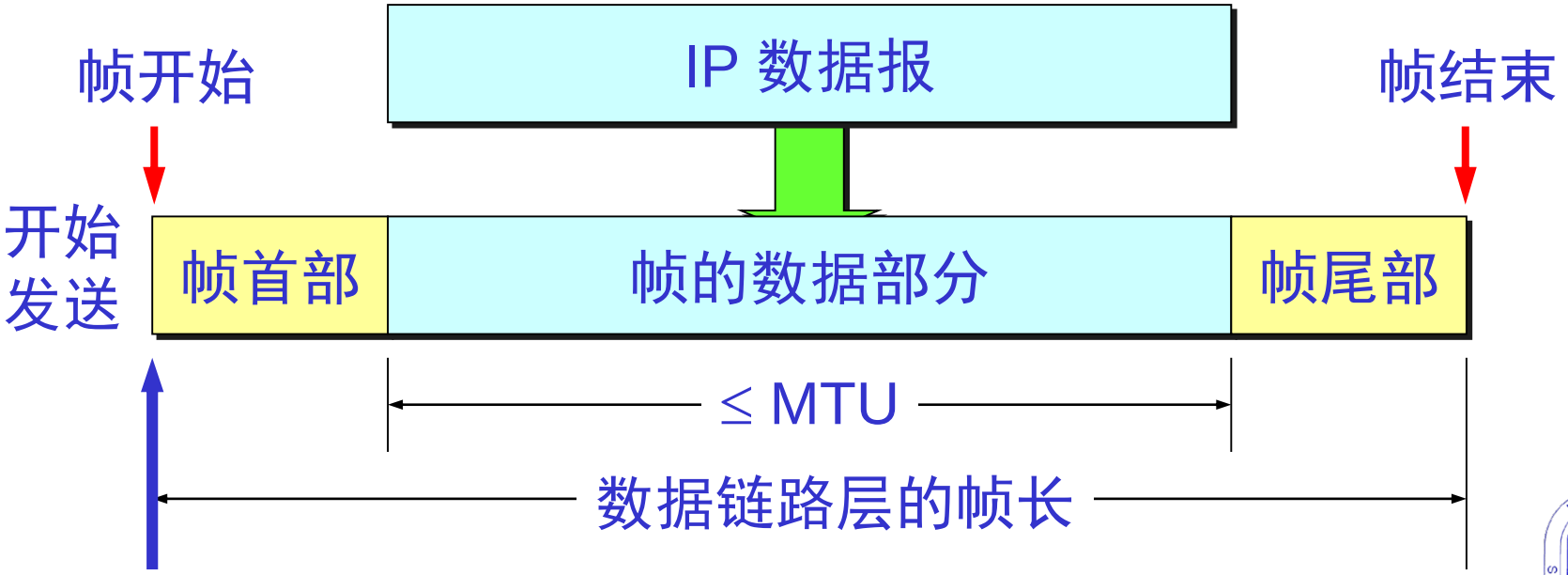
- 早期的数据通信协议曾叫作**通信规程**(procedure)。因此在数据链路层，规程和协议是同义语。

数据链路层的主要功能

1. 封装成帧（帧定界）
2. 透明传输
3. 差错控制
4. 流量控制
5. 寻址
6. 链路管理

1. 封装成帧

- **封装成帧**(framing)就是在一段数据的前后分别添加首部和尾部，然后就构成了一个帧。确定帧的界限。
- 首部和尾部的一个重要作用就是进行帧定界。

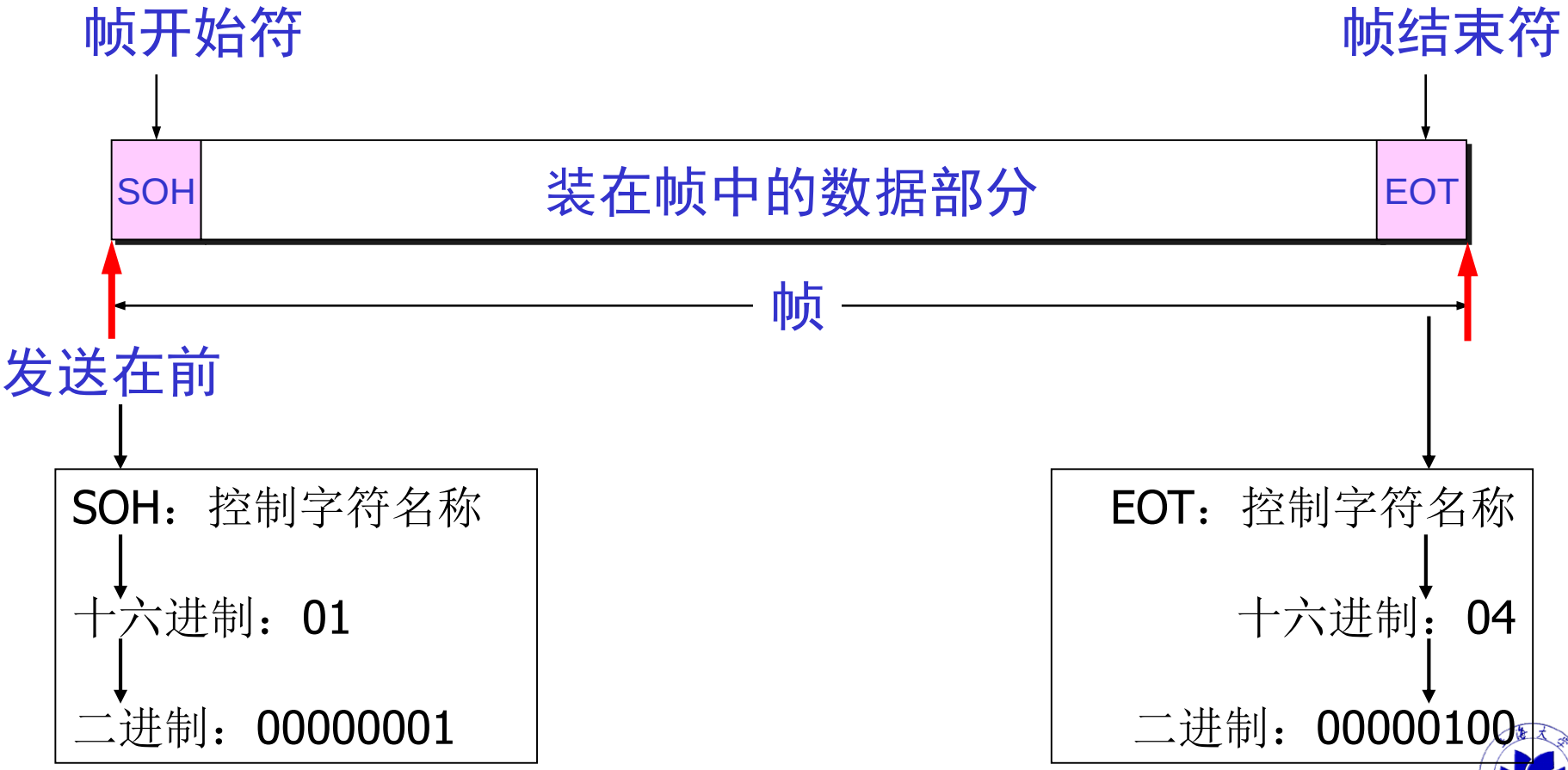


用控制字符进行帧定界的方法举例

ASCII控制字符，0 - 31，共32个

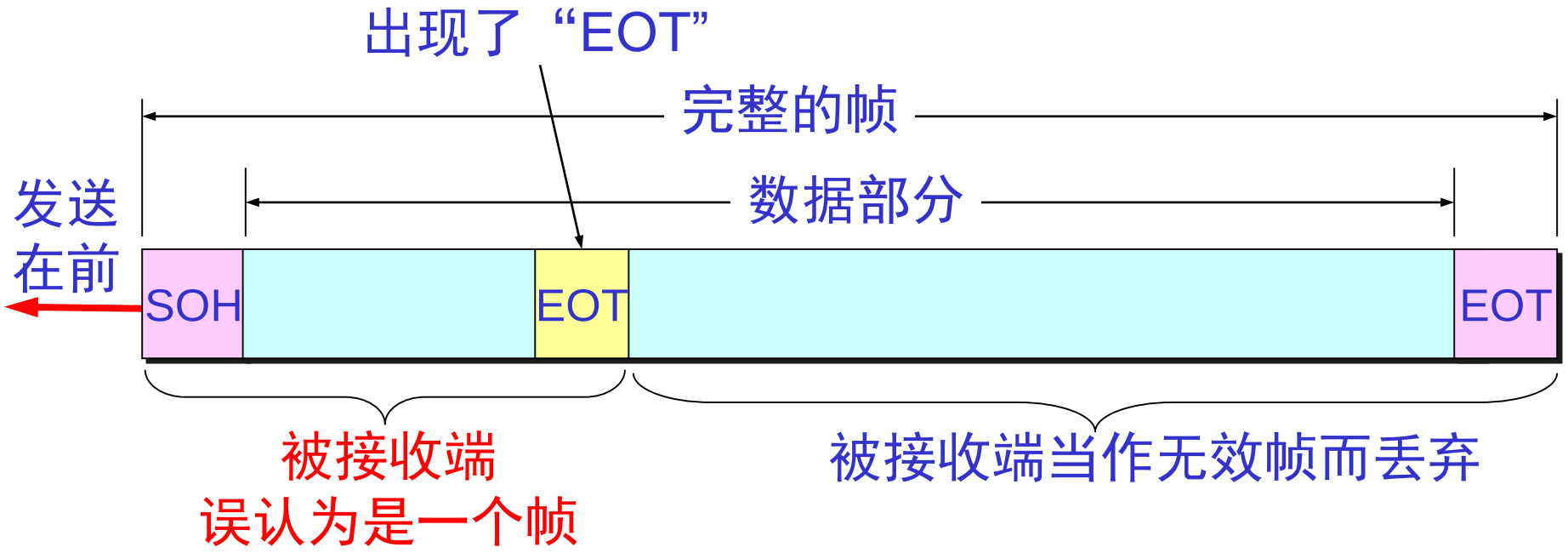
十进制	八进制	十六进制	二进制	符号	HTML	中文解释
DEC	OCT	HEX	BIN	Symbol	实体编码	Description
16	20	10	10000	DLE	&#016;	数据链路转义
17	21	11	10001	DC1	&#017;	设备控制 1
18	22	12	10010	DC2	&#018;	设备控制 2
19	23	13	10011	DC3	&#019;	设备控制 3
20	24	14	10100	DC4	&#020;	设备控制 4
21	25	15	10101	NAK	&#021;	拒绝接收
22	26	16	10110	SYN	&#022;	同步空闲
23	27	17	10111	ETB	&#023;	传输块结束
24	30	18	11000	CAN	&#024;	取消
25	31	19	11001	EM	&#025;	介质中断
26	32	1A	11010	SUB	&#026;	替换
27	33	1B	11011	ESC	&#027;	换码符
28	34	1C	11100	FS	&#028;	文件分隔符
29	35	1D	11101	GS	&#029;	组分隔符
30	36	1E	11110	RS	&#030;	记录分离符
31	37	1F	11111	US	&#031;	单元分隔符

用控制字符进行帧定界的方法举例



2. 透明传输

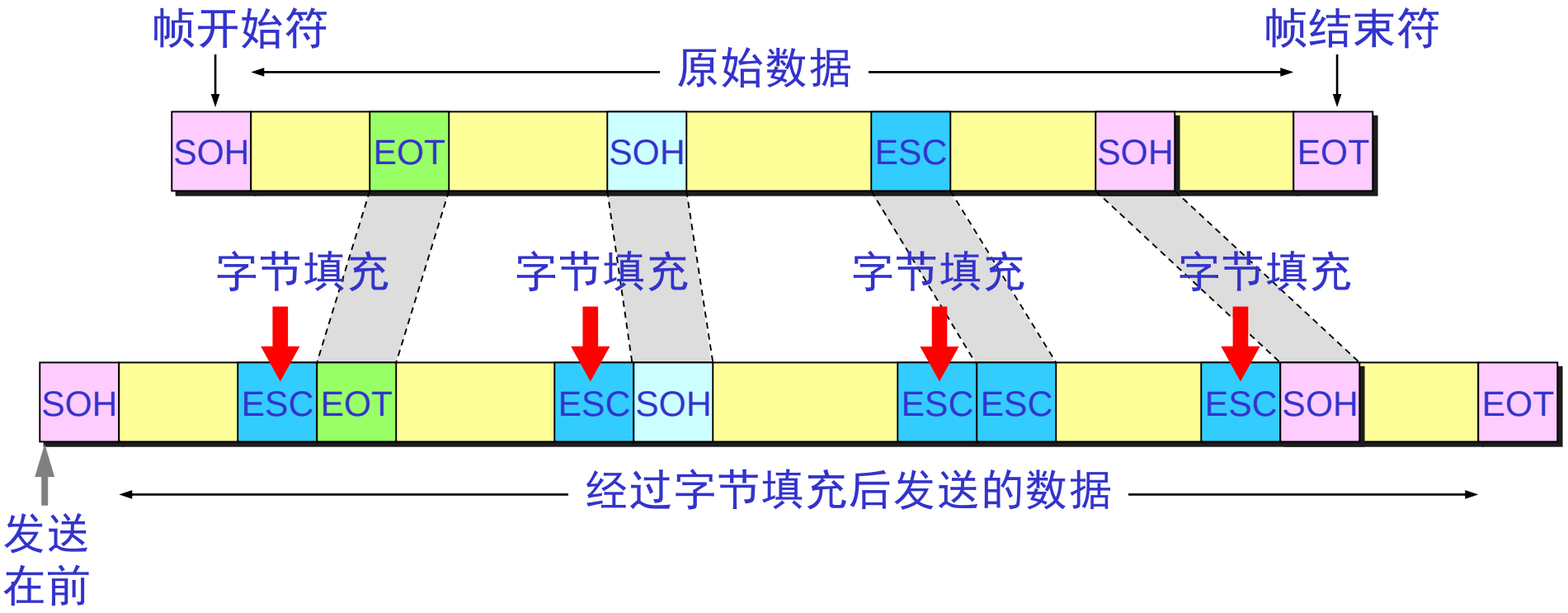
无论什么比特组合的数据都能通过数据链路层传输。



解决透明传输问题

- 发送端的数据链路层在数据中出现控制字符“SOH”或“EOT ”的前面插入一个转义字符“ESC”(其十六进制编码是 1B)。——字节填充(byte stuffing)或字符填充(character stuffing)
- 接收端的数据链路层在将数据送往网络层之前删除插入的转义字符。
- 如果转义字符也出现数据当中，那么应在转义字符前面插入一个转义字符。当接收端收到连续的两个转义字符时，就删除其中前面的一个。

用字节填充法解决透明传输的问题



3. 差错控制

(1) 比特差错

产生误码。

检错：用FCS（Frame Check Sequence,帧校验序列）

常用：CRC（Cyclic Redundancy Check,循环冗余码）

(2) 帧错

包括帧丢失、重复、失序

用帧编号、确认、重传机制

解决：反馈重发（ARQ，Automatic Repeat reQuest,

自动请求重发）

循环冗余检测CRC（Cyclic Redundancy Check）

- 是数据链路层传送的帧中广泛使用的差错检测技术。
- **检测过程：**
  - **（1）在发送端先把数据划分为组。假定每组k个比特。**

    如：假设待传送的一组数据 $M = 101001$（这里 $k = 6$）。
  - **（2）在M的后面添加供差错检测的n位冗余码。**

    如何得到冗余码？
    - 用二进制的模2运算进行 $2^n \times M$ 的运算，相当于在M后面添加n个0。
    - 得到的$(k+n)$位的数除以事先选定好的长度为$(n+1)$位的除数P，得出商是Q余数是R，余数R比除数P少1位，即R是n位。 **R为冗余码。**

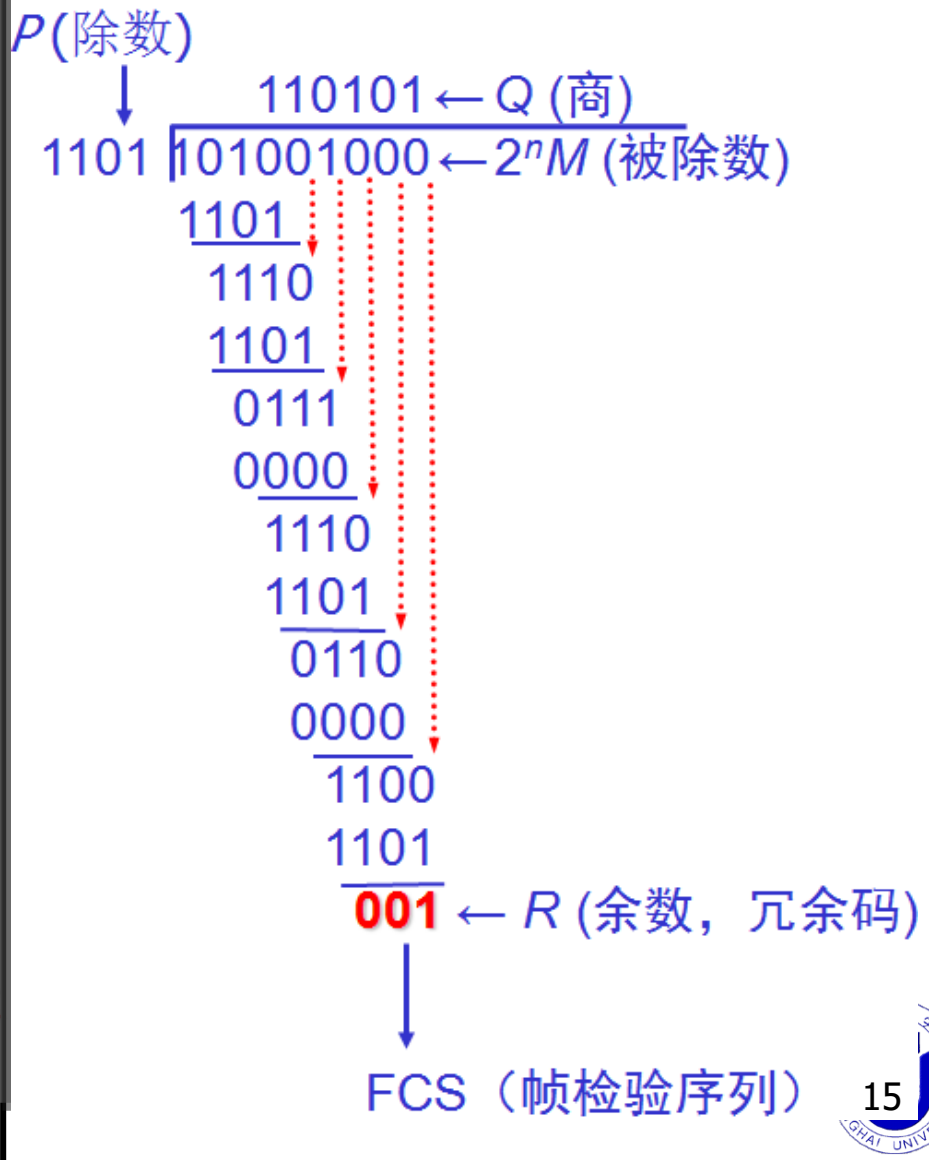
循环冗余检测CRC（Cyclic Redundancy Check）

- k=6， M=101001。
- 设n=3， 除数P=1101，
- 被除数是2^nM =101001000。
- 模2运算结果：

  商 Q = 110101，

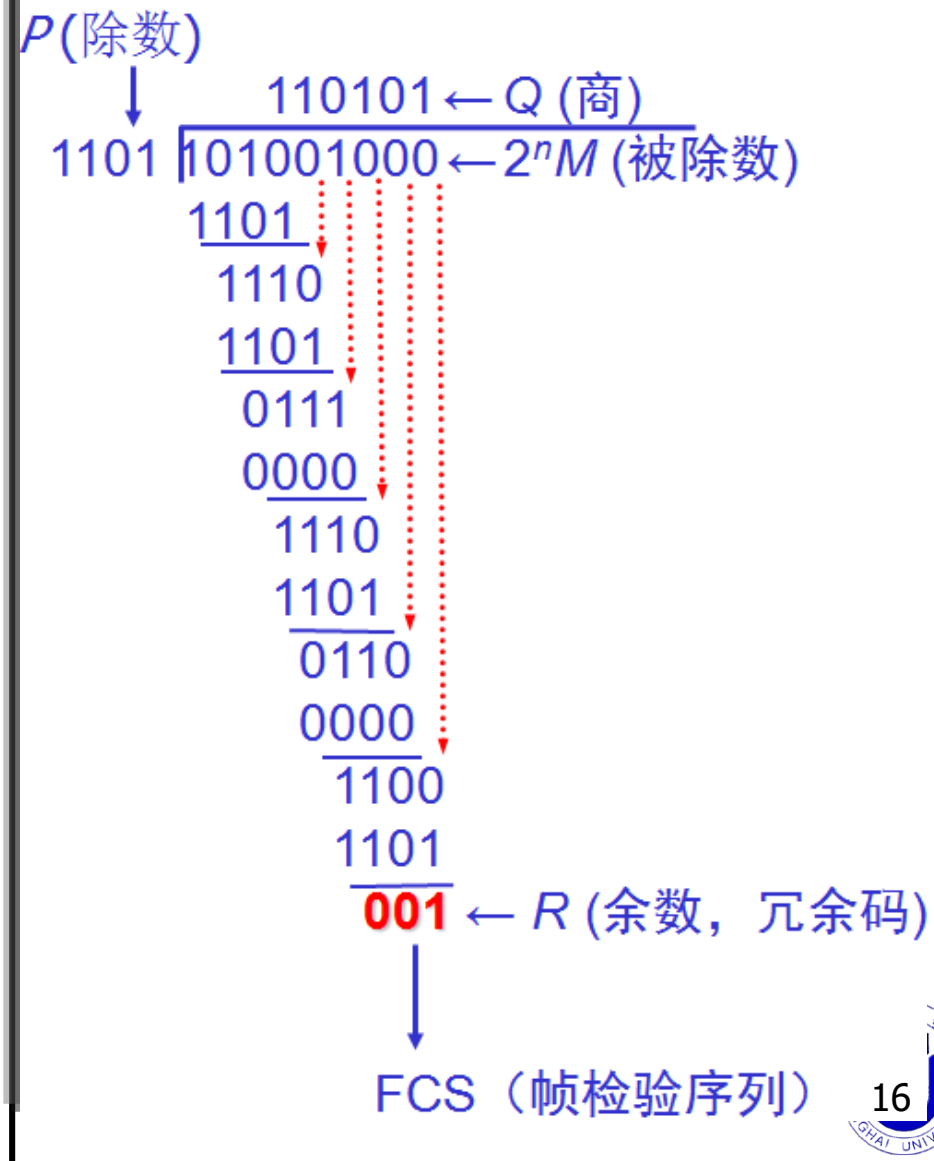
  余数 R = 001。
- 把余数R作为冗余码添加在数据 M 的后面发送出去。发送的数据是：2^nM + R，即：

  101001001，共(k+n)位。
- 在数据后面添加的冗余码称为帧检验序列 FCS (Frame Check Sequence)。



循环冗余检测CRC（Cyclic Redundancy Check）

- 循环冗余检验 CRC 和帧检验序列 FCS并不等同。
  - CRC 是一种常用的检错方法，而 FCS 是添加在数据后面的冗余码。
  - FCS 可以用 CRC 这种方法得出，但 CRC 并非用来获得 FCS 的唯一方法。



循环冗余检测CRC（Cyclic Redundancy Check）

- （3）接收端对收到的每一帧进行 CRC 检验
  - 把收到的每一帧都除以同样的除数P(模2)运算
    - 若得出的余数R =0，则这个帧没有差错，就接受(accept)。
    - 若余数 $R \neq 0$，则这个帧有差错，丢弃。

说明：

- 这种检测方法并不能确定究竟是哪一个或哪几个比特出了差错。
- 只要经过严格的挑选，并使用位数足够多的除数P，那么出现检测不到的差错的概率就很小很小。

应当注意

- 仅用循环冗余检验 CRC 差错检测技术只能做到无差错接受(accept)。
- “无差错接受”是指：“凡是接受的帧（即不包括丢弃的帧），我们都能以非常接近于 1 的概率认为这些帧在传输过程中没有产生差错”。
- 也就是说：“凡是接收端数据链路层接受的帧都没有传输差错”（有差错的帧就丢弃而不接受）。
- 要做到“可靠传输”（即发送什么就收到什么）就必须再加上确认和重传机制。

4. 流量控制

流量控制： 协调链路两端的发送/接收站之间的数据流量，以保证双方的数据吞吐量达到平衡的技术。

实质：**控制发送方的速率**

包括：**停等式流控**和**滑窗式流控**。

5. 寻 址

保证每一帧数据能送到正确的目的站，收方也应知道发送方是哪站。

6. 链路管理

数据链路连接的建立、维持和释放。

（面向连接的数据链路）

停止等待协议

流控：协调链路两端的发送/接收站之间的数据流量，以保证双方的数据吞吐量达到平衡的技术。

假设：

1、链路是理想信道，数据不出错，不丢失；

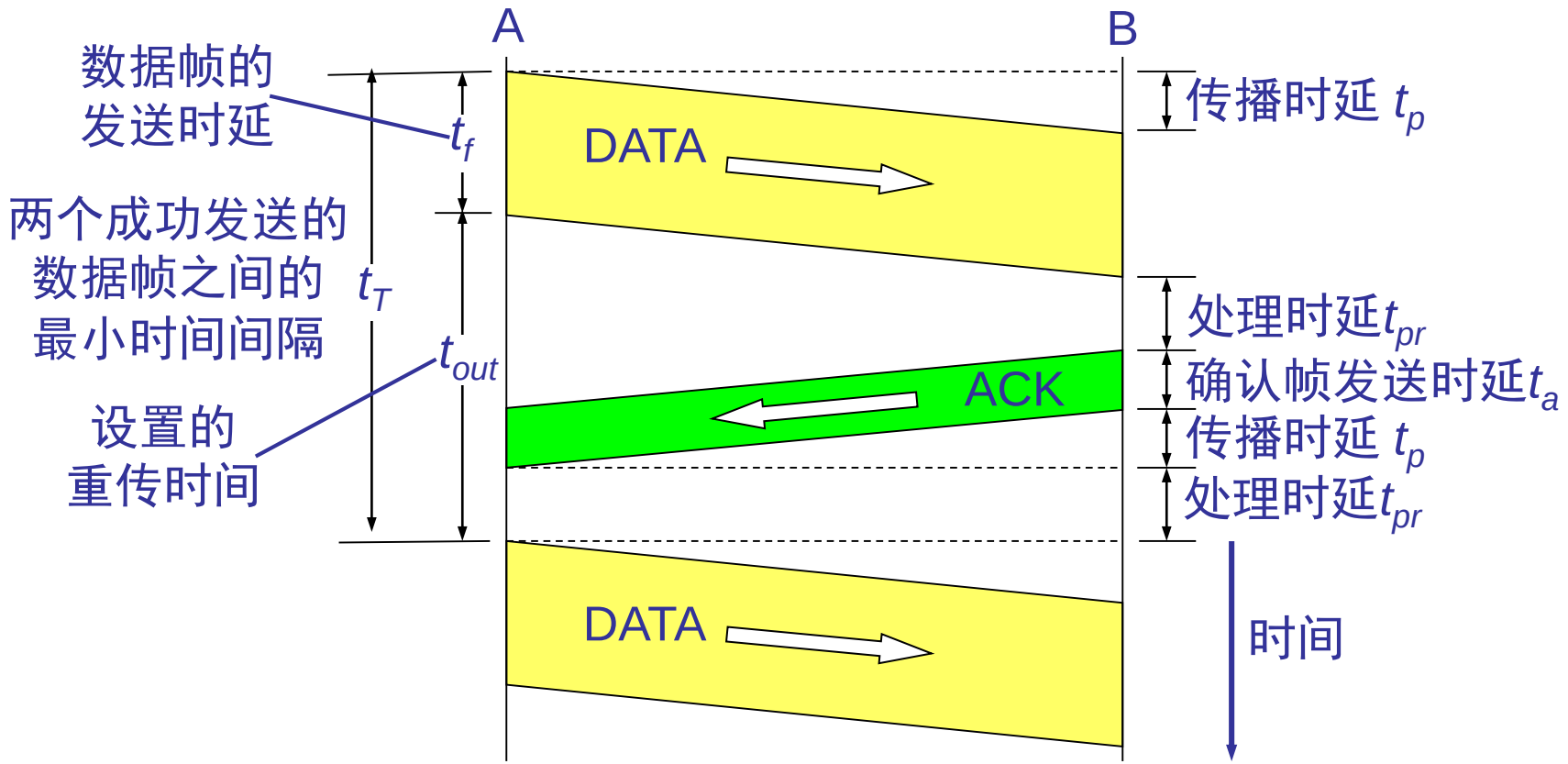
2、接收方总是来得及收下，并及时上交给主机。

工作原理：

发送端： 发送完一个数据帧后停止，等待接收端的响应帧（可以是字符或其它约定的代码）；

接收端： 如收到正确的数据帧，发ACK，否则发NAK（否定确定）。

停止等待协议中数据帧和确认帧的发送时间关系



$$\text{信道利用率} U = \frac{t_f}{t_T} = \frac{t_f}{t_f + 2t_p + 2t_{pr} + t_a}$$

停止等待协议

■无差错时信道利用率分析

如果忽略处理时延t_{pr}和响应帧发送时延t_a：

$$U=\frac{t_f}{t_f+2t_p}$$

其中：$t_f=\dfrac{\text{帧长}}{\text{数据发送速率}}=\dfrac{L}{C}$

$$t_p=\frac{\text{线路长度}}{\text{传播速度}}=\frac{d}{v}$$

定义：$a=\dfrac{t_p}{t_f}$(归一化时延指数)

则：$$U=\frac{t_f}{t_f+2t_p}=\frac{1}{1+2a}$$

停止等待协议

■无差错时信道利用率分析

例1： 双绞线长1000m，电磁波传播速度v=2×10⁸m/s，帧长L=1000bit，数据发送速率C=10Mbps。求信道利用率U。

$$t_f = \frac{L}{C} = \frac{1000bit}{10M} = 10^{-4}\text{s}$$

$$t_p = \frac{d}{v} = \frac{1000}{2\times10^8} = 5\times10^{-6}s$$

$$U = \frac{t_f}{t_f + 2t_p} = \frac{10^{-4}}{10^{-4} + 2\times5\times10^{-6}} = 90.9\%$$

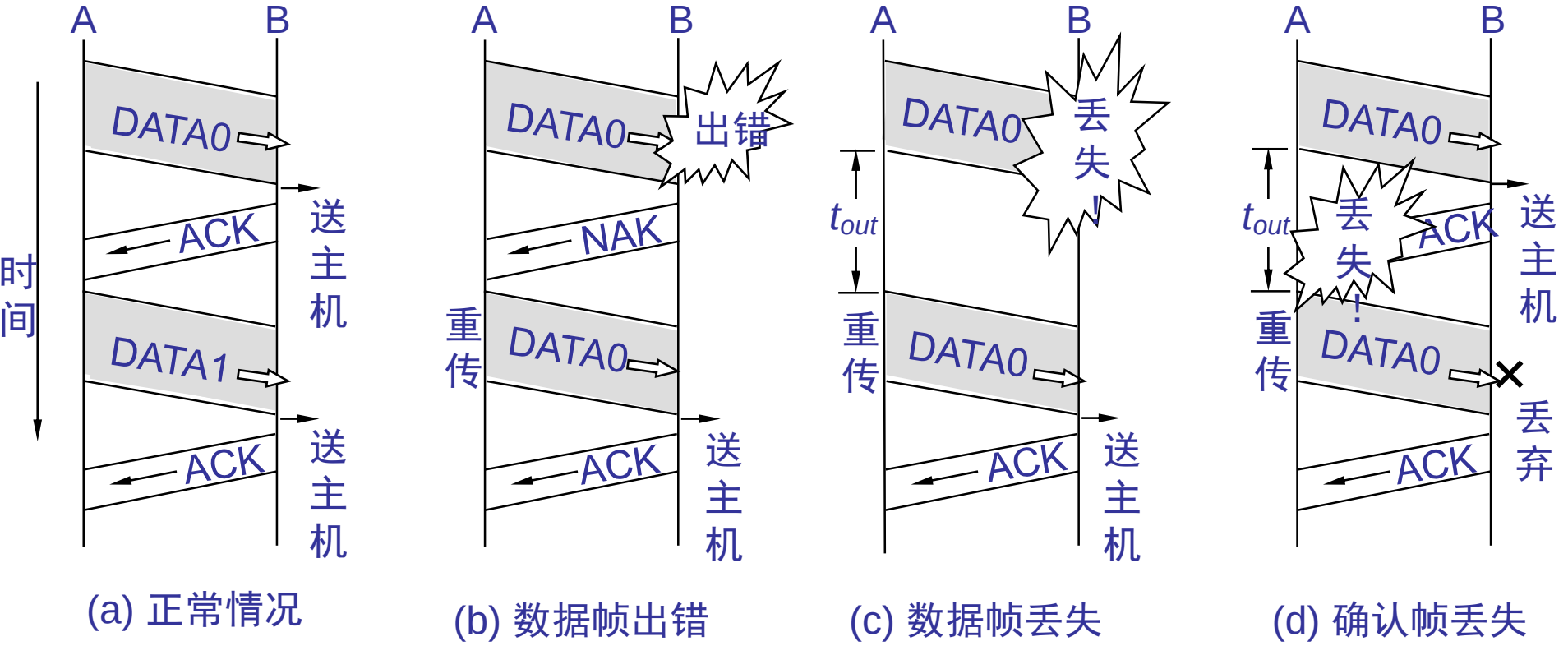
例2： 卫星线路的传播时延tp=270ms，帧长L=1000bit，数据发送速率C=64Kbps。求信道利用率U。

$$t_f = \frac{L}{C} = \frac{1000bit}{64K} = 15.62ms$$

$$U = \frac{t_f}{t_f + 2t_p} = \frac{15.62}{15.62 + 2\times270} = 2.81\%$$

实际的停止等待协议

四种情况



(a) 正常情况　(b) 数据帧出错　(c) 数据帧丢失　(d) 确认帧丢失

- 要解决的两个问题：
  - 丢帧后的系统恢复：计时器定时重传。
  - 防止重帧：对数据帧编号，响应帧也编号，帧序号周期为2，帧编号0或1。

停止等待协议

■有差错时信道利用率分析——停等式ARQ纠错技术

无差错时信道利用率：

$$U=\frac{t_f}{t_f+2t_p}=\frac{1}{1+2a}$$

■上式表明，若一次成功发送一个帧的时间t_f=1，则实际耗时为1+2a。

■如果传输出现差错，则发送N_r次（一次初发，N_r -1次重发）才成功，每次耗时1+2a，信道利用率：

$$U=\frac{1}{N_r(1+2a)}$$

■设误帧率为P，响应帧不出错，则数据帧发送i次才成功的概率P_i为：

$$P_i=P^{i-1}(1-P)$$

■平均发送次数：

$$\mathrm{Nr}=\sum_{i=1}^{\infty}iP_i=\sum_{i=1}^{\infty}iP^{i-1}(1-P)=\frac{1}{1-P}$$

$$U=\frac{1-P}{1+2a}$$

P=0时，即为无差错时的信道利用率

连续ARQ协议（滑窗协议）

- 在发送完一个帧后，不是停下来等待，而是在等待的同时继续发送数据帧。如果一次允许连续发送的帧数为n，则有：

$$U=\begin{cases}\dfrac{nt_f}{t_f+2t_p}=\dfrac{n}{1+2a} & (n<1+2a)\\ 1 & (n\geq 1+2a)\end{cases}$$

- n的取值与实际应用有关：
  - 当使用停等式协议时，$n=1$
  - 当使用地面传输链路时，通常$n=7$
  - 当使用卫星链路时，通常$n=127$

连续ARQ协议（滑窗协议）

窗口：是一张顺序号表，表中顺序是按模M（$M=2^n$）连续，用等分圆内可旋转而大小改变的扇形来表示滑动的窗口。

窗口最小者为下限，最大者为上限，包含的顺序号个数称为窗口的尺寸W。

连续ARQ协议（滑窗协议）

发送窗口W_S：

允许连续发送的帧的顺序号表，落入窗口内的序号表示允许连续发送的所有帧的顺序号。

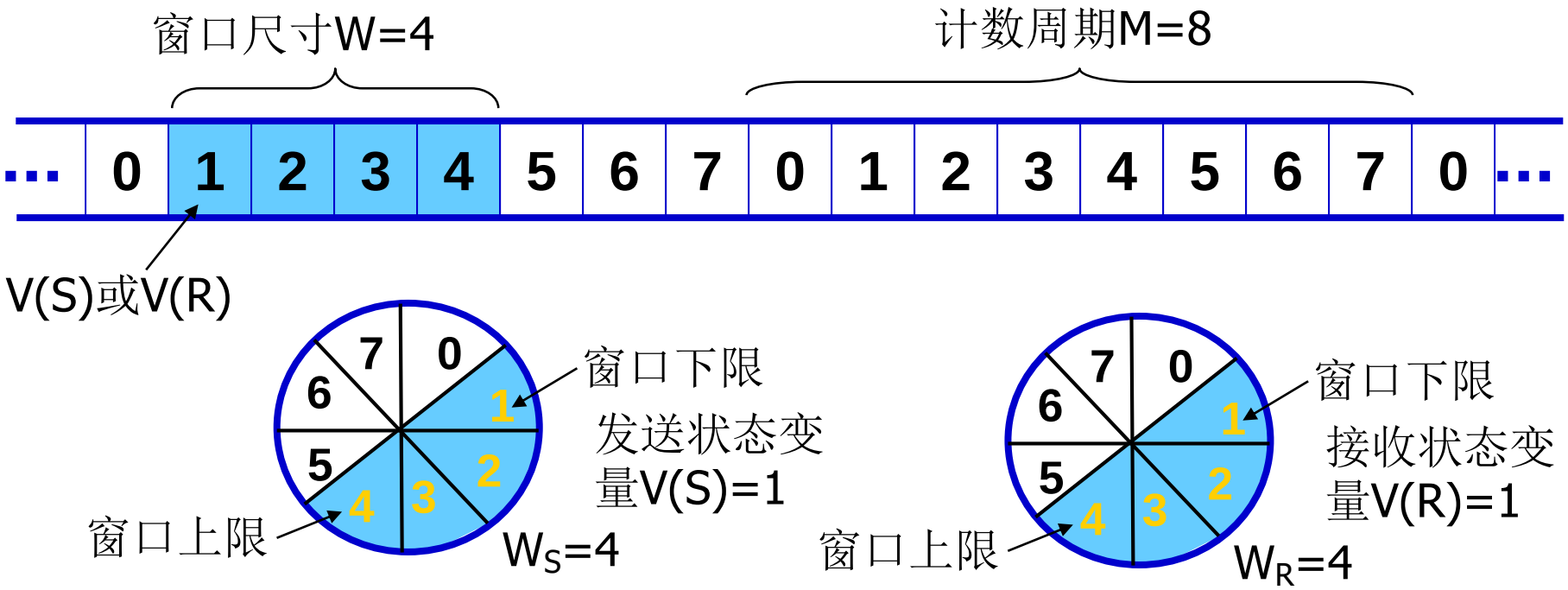
窗口滑动： 发送完一个帧，窗口下限推进一个序号。收到一个确认帧，窗口 的上限推进一个序号。

接收窗口W_R：

允许连续接收的帧的顺序号表，落入窗口内的序号表示允许连续接收的所有帧的顺序号。

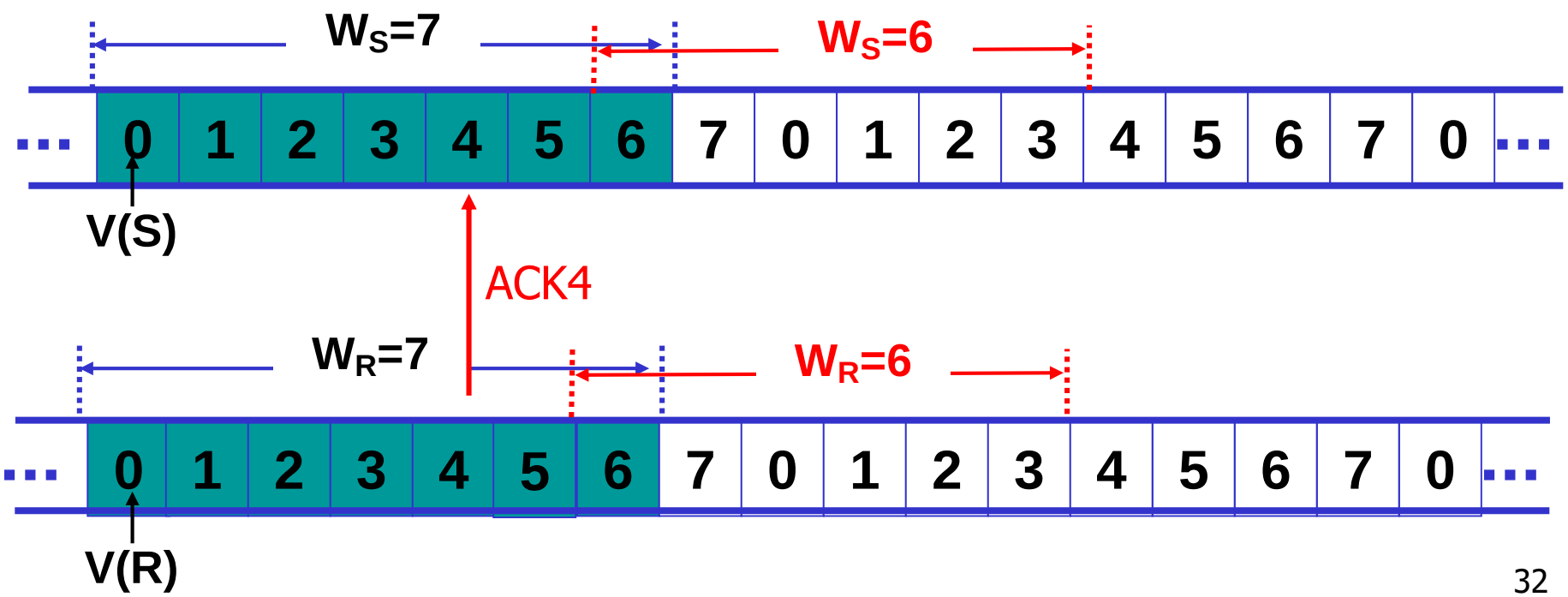
窗口滑动： 接收一个帧，窗口下限推进一个序号。发出一个帧的确认，窗口 的上限推进一个序号。

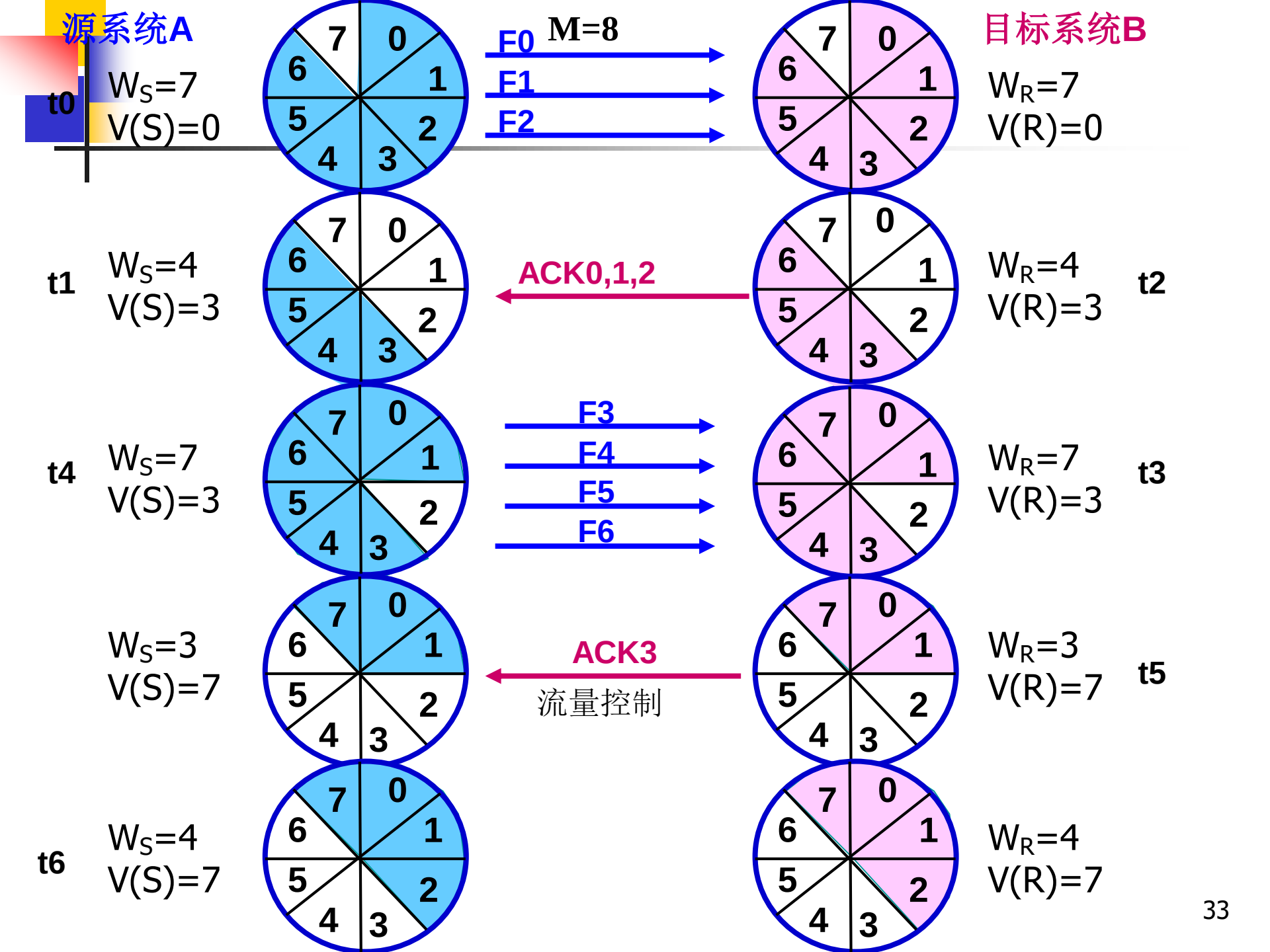
连续ARQ协议（滑窗协议）



连续ARQ协议（滑窗协议）

- **滑窗原理**：
- **发送窗口W_S**：发送完一个帧，窗口下限推进一个序号；收到一个确认帧，窗口上限推进一个序号。
- **接收窗口W_R**：接收完一个帧，窗口下限推进一个序号；发出一个确认帧，窗口上限推进一个序号。



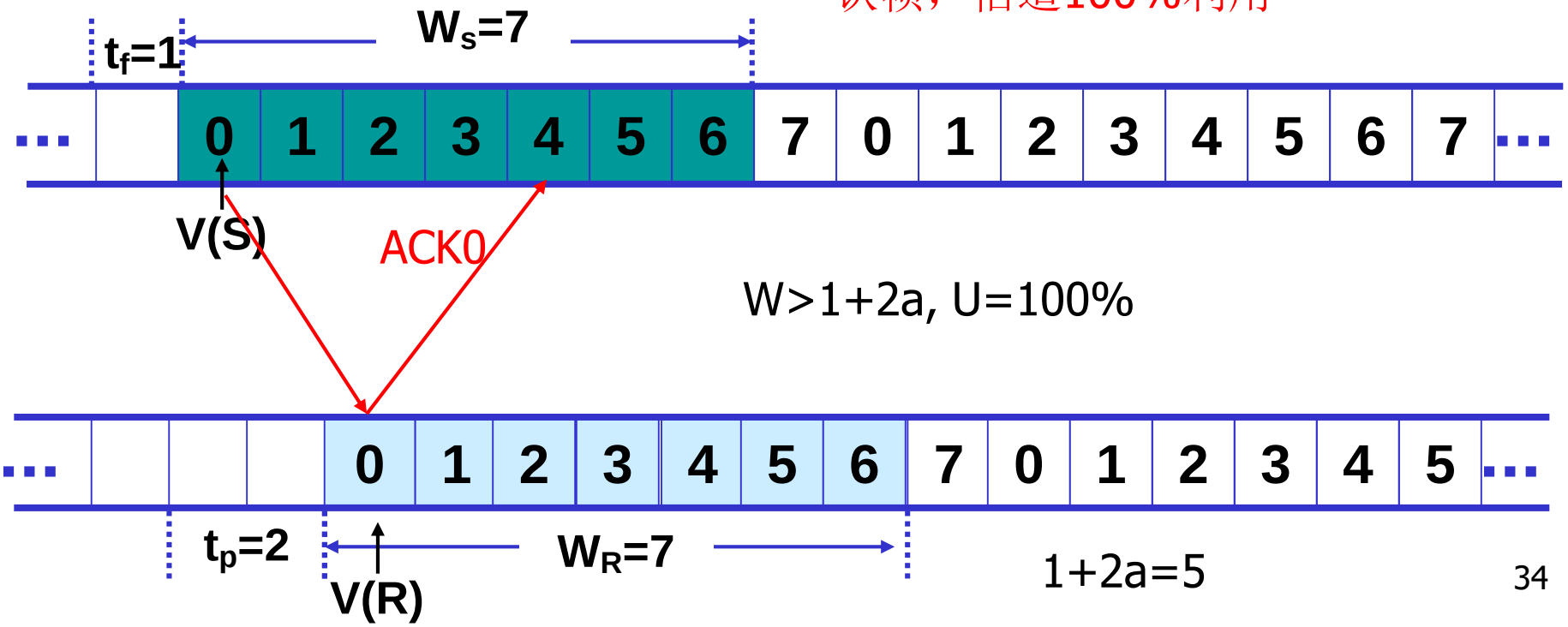

源系统A
目标系统B
M=8
t0
W_S=7
V(S)=0
F0
F1
F2
W_R=7
V(R)=0
t1
W_S=4
V(S)=3
ACK0,1,2
W_R=4
V(R)=3
t2
t4
W_S=7
V(S)=3
F3
F4
F5
F6
W_R=7
V(R)=3
t3
W_S=3
V(S)=7
ACK3
流量控制
W_R=3
V(R)=7
t5
t6
W_S=4
V(S)=7
W_R=4
V(R)=7

连续ARQ协议（滑窗协议）

- **信道利用率分析**：假设无流控，无传输差错。

$$U = \frac{Wt_f}{t_f + 2t_p} = \frac{W}{1+2a} \quad \Rightarrow \quad U=1 \quad (W \geq 1+2a)$$

发送站窗口缩小到0之前一定收到确认帧，信道100%利用

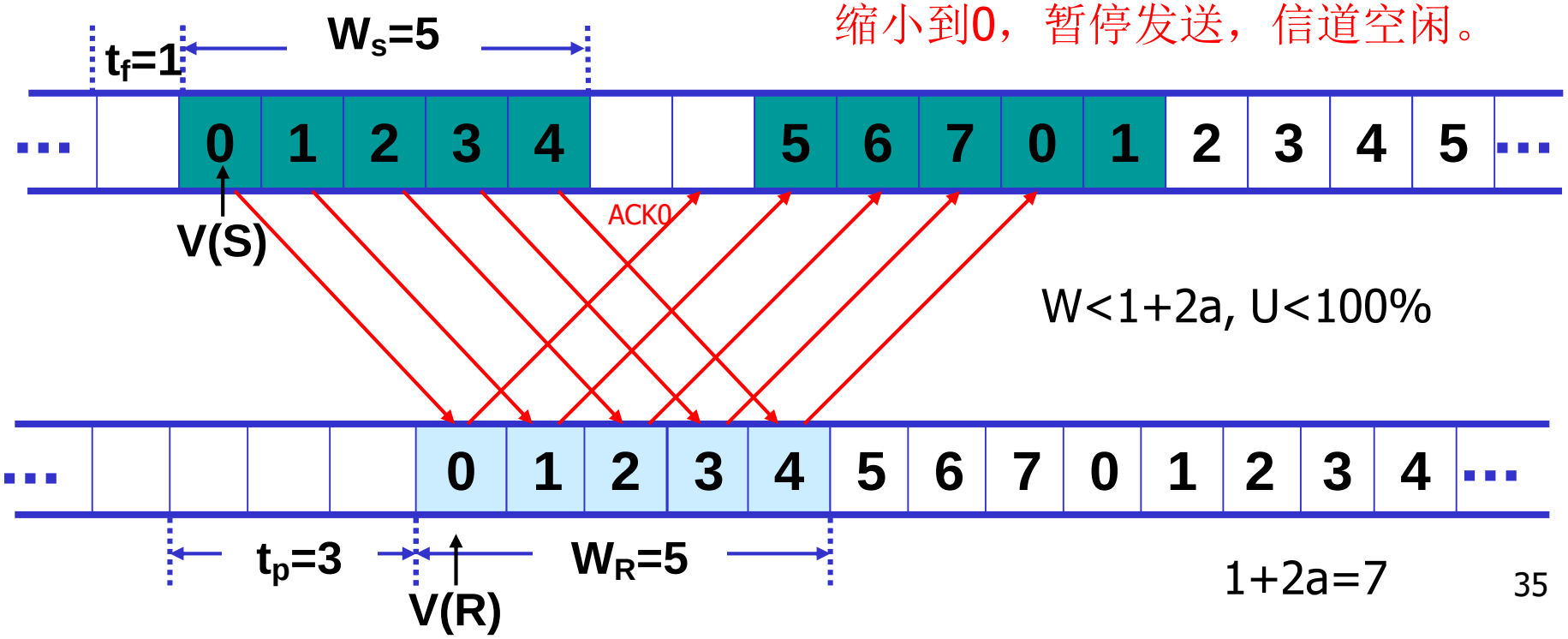


连续ARQ协议（滑窗协议）

- **信道利用率分析**：假设无流控，无传输差错。

$$U=\frac{Wt_f}{t_f+2t_p}=\frac{W}{1+2a} \quad \Rightarrow \quad U=\frac{W}{1+2a} \quad (W<1+2a)$$

发送站收到确认帧之前，窗口已经缩小到0，暂停发送，信道空闲。



连续ARQ协议（滑窗协议）

- **返回N组连续ARQ纠错技术（GBN-ARQ）**

GBN-ARQ——Go-Back-N Automatic Repeat Request

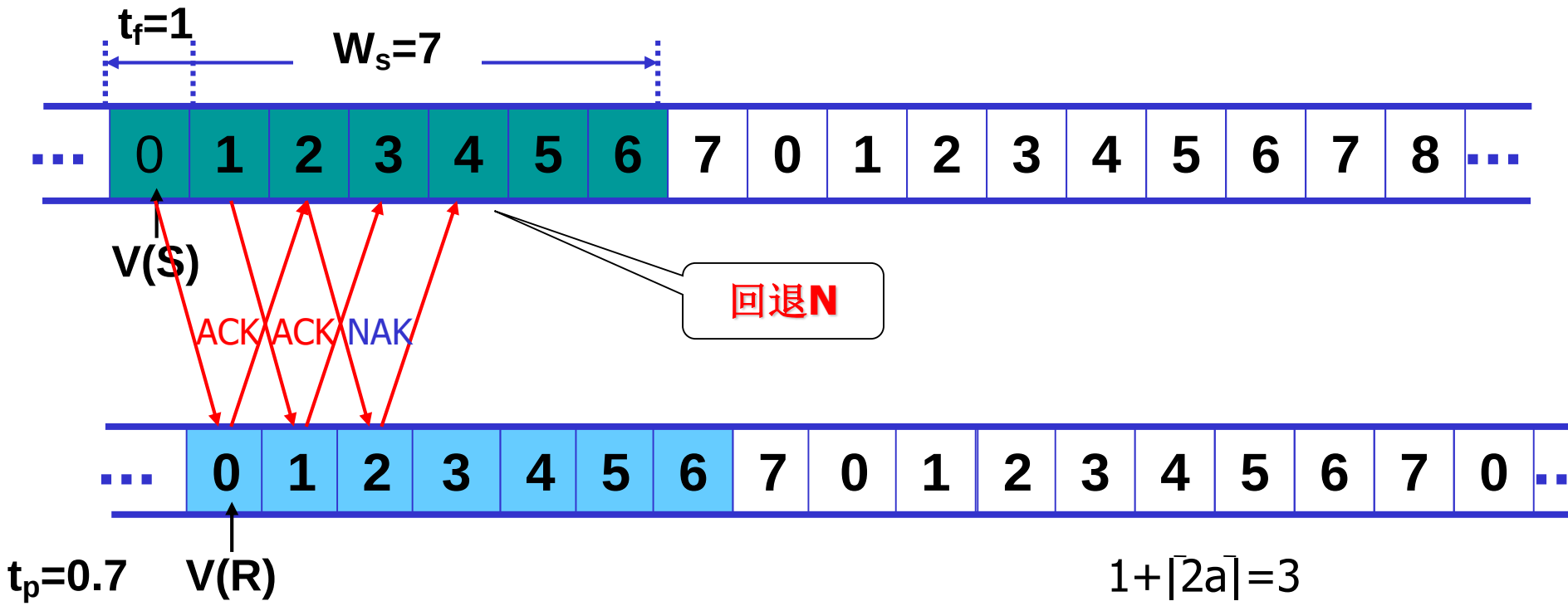
按窗口大小连续发送和接收。

- **接收站**：发现错误时，回送一个NAK响应帧，并从该帧开始连续删除后面的N帧数据，再继续接收。

- **发送站**：发现NAK时，将发送窗口下限回退N个序号，重发这N个数据帧。

连续ARQ协议（滑窗协议）

- 返回N组连续ARQ纠错技术（GBN-ARQ）



- **优点**：简单，易于实现；
- **缺点**：效率低。
- **累计确认**：ACK（2）

$$N=1+\lceil 2a \rceil =3$$

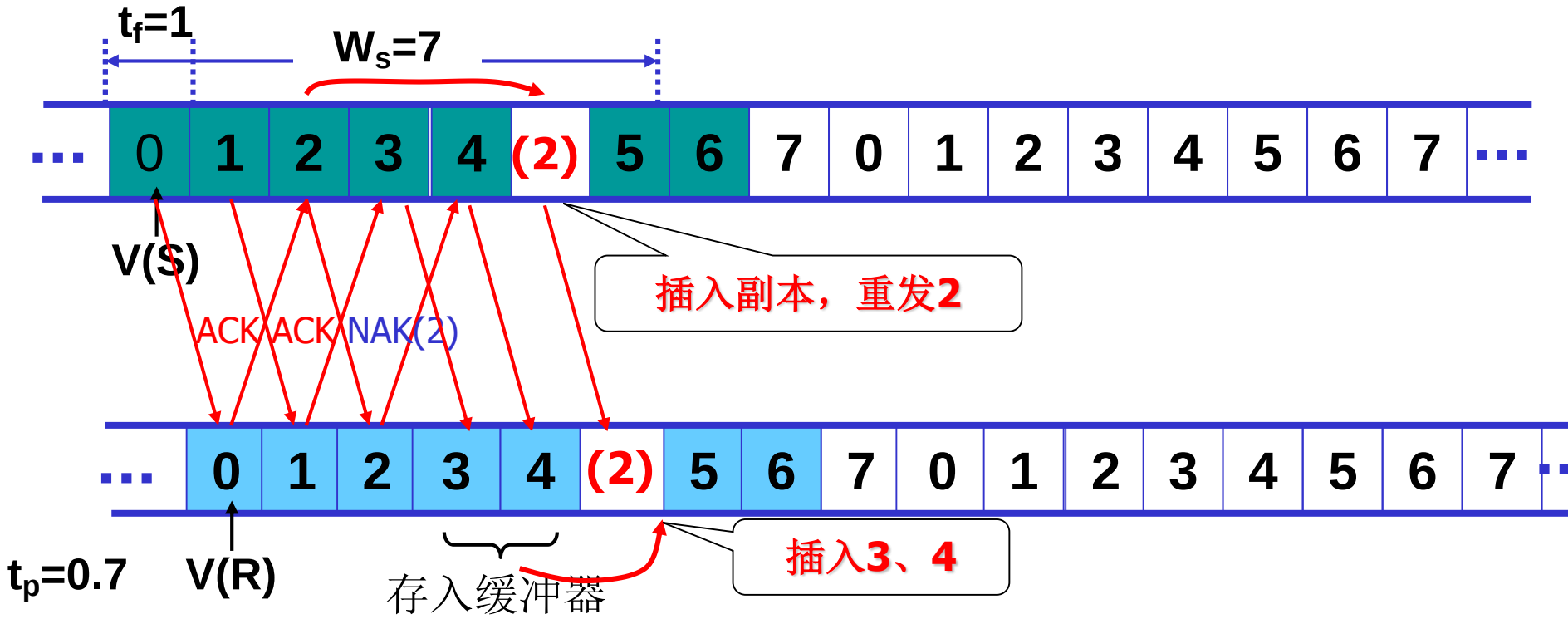
连续ARQ协议（滑窗协议）

- **选择重发连续ARQ纠错技术（SRP-ARQ）**

按窗口大小连续发送和接收，接收站的响应帧携带出错帧的序号。当发送站收到NAK时，从存储器中取出副本，插入到队列前面重发该帧。

连续ARQ协议（滑窗协议）

■选择重发连续ARQ纠错技术（SRP-ARQ）



■优点：提高了效率；

■缺点：接收端需要消耗较多的存储空间。控制机制复杂（暂存、排序等操作）花费较多处理时间。

■因此，SRP-ARQ不如GBN-ARQ应用广泛。特别是在传播时延较小的信道中。

思考题

- **返回N组连续ARQ协议**采用的帧号位数为3，窗口尺寸为4，请画出下列事件发生时输入输出端的滑动窗口图：

(a)初始态；　　(b)发送帧0，1，2；

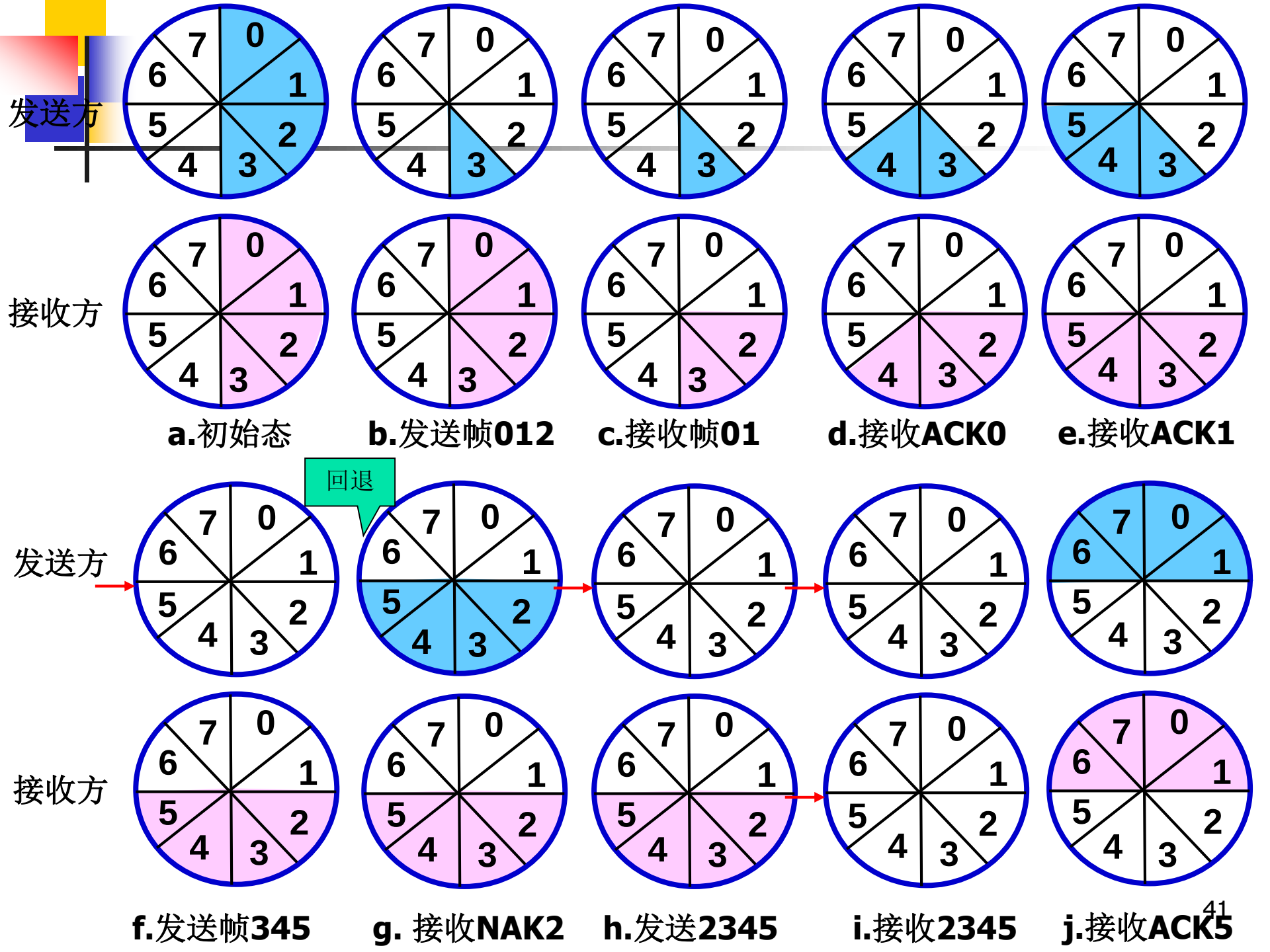
(c)接收帧0，1；　　(d)接收ACK0；

(e)接收ACK1；　　(f)发送帧3，4，5；

(g)接收NAK2；　　(h)发送帧2，3，4，5；

(i)接收帧2，3，4，5；

(j) 接收ACK5


发送方
接收方
a.初始态
b.发送帧012
c.接收帧01
d.接收ACK0
e.接收ACK1
回退
发送方
接收方
f.发送帧345
g. 接收NAK2
h.发送2345
i.接收2345
j.接收ACK5